6× Smaller KV Cache · **8× Faster Attention** · **No Retraining**

TurboQuant

KV Cache Compression for LLM Inference

A Structured Practical Learning Guide

Based on Google Research / ICLR 2026 paper · arXiv:2504.19874

Compiled by **Douglas Mun** with AI assistance

TLP:CLEAR — Public, Unrestricted Distribution

6×	8×	3.5 bits	~0%
KV cache reduction	Attention speedup (H100)	Effective precision	Accuracy loss (NiH test)

Table of Contents

Disclaimer

TLP:CLEAR — Public, Unrestricted Distribution

This document is classified TLP:CLEAR and may be shared freely without restriction. No prior approval is required for distribution.

This guide is provided for educational and informational purposes only. It synthesises publicly available research, community findings, and first-hand implementation experience as of April 2026. All technical claims should be independently verified against the sources cited in Section 8. Benchmark figures are reproduced from community implementations and may differ from official results.

Code examples and CLI commands reflect nightly builds as of April 2026. Flags, package names, and APIs are subject to change. Always run `vllm-mlx serve --help` to verify available flags for your specific installed version before use in production.

The authors of referenced papers and community implementations retain all intellectual property rights to their respective works. This document is not affiliated with or endorsed by Google, Anthropic, or any other organisation.

1. What is TurboQuant?

TurboQuant was uploaded to the arXiv on April 28, 2025 (arXiv:2504.19874) and was accepted to ICLR 2026. It gained widespread attention in early 2026 when community implementations went viral — SK Hynix dropped 6%, Samsung fell 5%, Micron slid 3.4% — and Cloudflare CEO Matthew Prince called it "Google's DeepSeek moment". The paper presents a training-free, data-oblivious vector quantization algorithm that compresses the Key-Value (KV) cache of large language models to ~3.5 effective bits per value. The result: up to 6× reduction in KV cache memory and up to 8× speedup on NVIDIA H100 GPUs for attention computation — all without measurable accuracy degradation.

Within 24 hours of gaining viral attention, independent developers had working implementations on GitHub. Within three weeks, the core algorithm was merged into mainstream vLLM (April 15, 2026).

1.1 What TurboQuant Is — and Is Not

TurboQuant is only a KV cache compressor. It operates exclusively on the runtime KV cache during inference. It is **not** a weight quantizer (that's GPTQ, AWQ, GGUF). It doesn't reduce model size, prefill compute, or parameter count. Think of it as a near-lossless quantization layer that sits between your attention mechanism and VRAM — it introduces tiny, mathematically bounded distortion in exchange for 4-6× compression.

- **Does:** Compresses stored key and value vectors as they are written to cache
- **Does:** Decompresses them when attention reads them back during decoding
- **Does:** Works on any transformer model with no calibration data or retraining
- **Does not:** Reduce model size or weight memory footprint
- **Does not:** Compress prefill computation or reduce FLOPs
- **Does not:** Replace weight quantization — both stack independently

1.2 Who Benefits

TurboQuant is directly relevant if any of these apply to your deployment:

- Long-context inference with windows of 50,000+ tokens
- Multi-turn agentic workflows that accumulate KV cache across tool calls
- High-concurrency serving (100+ simultaneous users)
- RAG pipelines are processing many retrieved document chunks
- Memory-constrained hardware where the KV cache is the VRAM bottleneck

For short conversational inference under 8k tokens with low concurrency, the benefit is modest — KV cache isn't your constraint there

2. The Memory Wall TurboQuant Targets

Most conversations about LLM efficiency focus on model weights. A 70B-parameter model in FP16 consumes approximately 140 GB of VRAM — a fixed cost, loaded once and held for the session. But the KV cache is dynamic: it grows with every token processed, every layer, and every concurrent user.

2.1 The KV Cache Math

Cache memory = layers × seq_len × 2 (K + V) × num_KV_heads × head_dim × 2 bytes (FP16) × batch_size

Note: Modern GQA models use fewer KV heads than attention heads. For Llama-3.1-70B: 80 layers, 8 KV heads, 128 head_dim — the KV dimension is 8 × 128 = 1,024, not the full hidden dimension of 8,192.

> Example: Llama-3.1-70B (80 layers, 8 KV heads, head_dim 128) at 128,000 token context:80 × 128,000 × 2 × 8 × 128 × 2 bytes = ~41.9 GB KV cache — before model weights. At 10 concurrent users, that is ~419 GB. At 512 users: ~2.1 TB.

This forces engineers into multi-GPU configurations, not because the model needs more compute, but because the inference memory footprint overflows a single accelerator's capacity. The cache grows linearly with the sequence length, layer count, and batch size.

2.2 Why Prior Compression Methods Were Insufficient

Prior methods fell into two categories, each with fundamental limitations:

- **Scalar quantization:** Methods like FP8 and KIVI reduce bit-width but struggle with outliers. The top 1% of KV values may be 10–100× larger than the median, causing linear quantization to crush the precision of normal tokens. KIVI achieved only ~2.6× compression.
- **Token pruning:** SnapKV and PyramidKV discard "less important" tokens — but you can't know which tokens will matter later. This introduces irreversible information loss for long-range dependencies.

TurboQuant approaches the problem from a completely different geometric direction: it changes the statistical properties of the vectors before quantization, rather than working around them after.

3. How TurboQuant Works: The Two-Stage Algorithm

In the paper, TurboQuant is a two-stage pipeline. The first stage (Algorithm 1, often called "PolarQuant" in community implementations) is a rotation-then-quantize step. The second stage adds a 1-bit Quantized Johnson-Lindenstrauss (QJL) residual correction. The community has found that Stage 1 alone is what works in production — see Section 3.2. The core architectural insight is geometric: KV cache vectors are hard to quantize efficiently because their value distributions are non-uniform and correlated across dimensions. TurboQuant removes both problems before quantization begins.

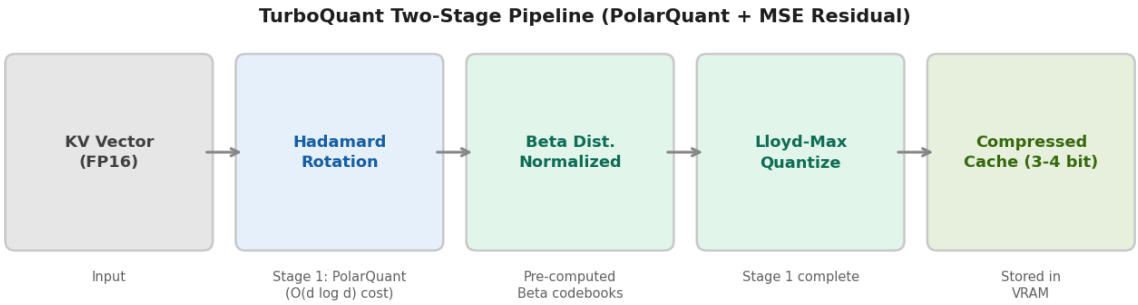

Figure 1: TurboQuant two-stage pipeline for KV cache compression

3.1 Stage 1: Rotation + Quantization ("PolarQuant")

Naming note: The term "PolarQuant" comes from community blog posts and implementations. The original paper (arXiv:2504.19874) refers to this as "TurboQuant_mse" — Algorithm 1. The paper's full method, which combines Algorithm 1 with the QJL residual, is called "TurboQuant". In production, skip QJL and use Algorithm 1 only (see Section 3.2).

Step 1: Random Orthogonal Rotation (Hadamard Transform)

Each key and value vector is multiplied by a random orthogonal rotation built from the randomized Hadamard transform (a Hadamard matrix combined with a random ±1 sign vector). This costs O(d log d) — not O(d²) for a dense rotation — making it practical at LLM head dimensions of 64–256.

Key property: A random orthogonal rotation is norm-preserving. It does not change the length of any vector, nor inner products between pairs of vectors that are rotated by the same matrix, so attention dot products survive intact when query, key, and value are all rotated together. What it does change is the distribution of energy across dimensions.

Step 2: Beta Distribution Normalization

After rotation, each coordinate of the vector independently follows an approximately Beta distribution — regardless of what the original distribution looked like. This is the mathematical guarantee that makes TurboQuant work: the rotation induces a known, predictable Beta-like distribution on every coordinate, regardless of the input data — so a single set of pre-computed quantization codebooks works universally.

Step 3: Pre-computed Lloyd-Max Codebooks

Because the post-rotation distribution is Beta and is analytically known, optimal quantization buckets can be computed in advance using the Lloyd-Max algorithm. These codebooks are baked in as compile-time constants. There is no per-model calibration, no dataset passes, and no warm-up required — PolarQuant works out of the box on any transformer architecture.

PolarQuant also eliminates the per-block scaling factors that INT4 methods must carry in compressed values. Those scaling factors partially defeat compression at low bit-widths. By converting to polar-like coordinates, PolarQuant removes this overhead entirely.

3.2 Stage 2: QJL — Theory vs Practice

The paper adds a second stage: Quantized Johnson-Lindenstrauss (QJL), a 1-bit error-correction pass designed to remove systematic bias in inner-product estimates after PolarQuant.

Community finding (5+ independent implementation groups): QJL hurts attention quality in practice, despite being mathematically correct for raw inner products.Reason: Attention runs dot products through softmax, which exponentially amplifies variance. QJL reduces bias but introduces random noise — and softmax magnifies it. Lower-variance biased quantization (MSE-only) wins after softmax. Recommendation: Use PolarQuant + MSE-only quantization for KV cache. Skip QJL. (QJL is correct for vector-search applications where no softmax is applied.)

4. Benchmark Results

TurboQuant's headline numbers are unusual in applied ML: near-lossless compression at 6× with no retraining, hardware speedup, and mathematically provable distortion bounds — all simultaneously.

4.1 LongBench and Needle-in-Haystack

Method	Bits	Compression	LongBench	Needle-in-Haystack
FP16 Baseline	16	1×	50.10	1.000
TurboQuant	**3.5 (eff.)**	**~6×**	**50.06**	**0.997**
TurboQuant	2.5 (eff.)	~8×	49.44	0.997
KIVI-3	3 (storage)	~2.6×	48.50	0.981
SnapKV	N/A (pruning)	~2×	44.57	0.858

Table 1: TurboQuant vs prior KV cache compression methods on standard benchmarks

Reconciling the headline numbers: The paper's 6× / 8× claims compare TurboQuant's effective bit-width (3.5 / 2.5 bits) against bf16 storage (16 bits), giving 16/3.5 ≈ 4.6× and 16/2.5 = 6.4× — the higher ratios additionally credit the elimination of per-block scaling-factor overhead that INT4 methods carry. Real-world vLLM presets (Section 5.2) achieve 2.6×- 4.9× performance gains, depending on the configuration. Treat 6× as best-case marketing; 3.8× (turboquant_4bit_nc) is the realistic production figure.

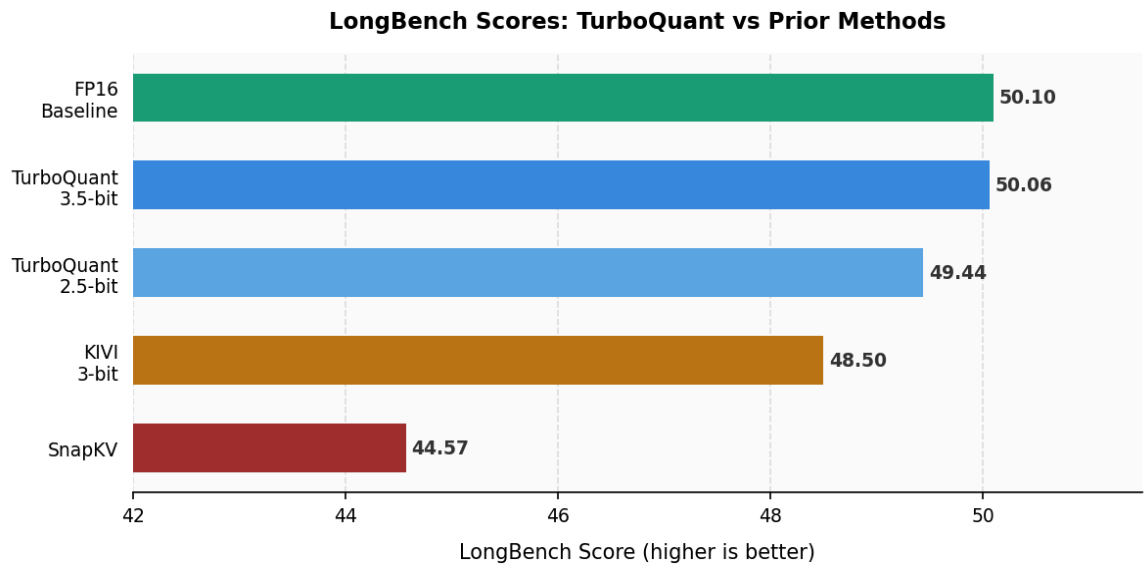

Figure 2: LongBench scores — TurboQuant maintains FP16 parity, where KIVI and SnapKV degrade

4.2 Community-Validated Sweet Spots

The paper's theoretical claims hold, but community experiments have refined which configurations work in practice:

4-bit: Quality is essentially indistinguishable from FP16 on models of 3B+ parameters. This is the recommended default for production use.

3-bit: Noticeable degradation on models smaller than 8B. Acceptable for larger models where the compression benefit outweighs the slight quality reduction.

2-bit and below: Produces garbage output without QJL residual correction — and QJL has its own attention-softmax issues (see Section 3.2). Avoid unless you have a specific research reason.

Small models (0.5B-1.6B): Quantization noise can produce repetitive or degraded output even at 3-bit. Always evaluate carefully on your target model size.

5. Code & Setup

5.1 Implementation Status (April 2026)

Framework	Status	How to Use	Maturity
vLLM (CUDA)	Merged upstream (PR #38479, April 15 2026)	Use --kv-cache-dtype turboquant_4bit_nc preset	Production-ready
vllm-mlx (Apple Silicon)	Nightly build only	Use --kv-cache-quantization + --kv-cache-quantization-bits 4	Stable
llama.cpp	Community implementation available	Built from a community fork with TQ support	Stable
turbokv (Python)	pip install turbokv	patch_model_for_turbokv(model, bits=4) — drop-in for HF	Stable
SGLang	Feature request open (Issue #21618)	Not yet available	In progress
Official Google	Expected Q2 2026	Reference implementation pending	Pending

Table 2: TurboQuant implementation status across inference frameworks

5.2 vLLM Presets

TurboQuant is available in stock vLLM via named presets on the --kv-cache-dtype flag. Choose your preset based on model size and quality requirements:

Preset (--kv-cache-dtype)	Keys	Values	Compression	PPL Impact	Use Case
`turboquant_k8v4`	FP8	4-bit	2.6×	+1.17%	Safest, near-lossless
`turboquant_4bit_nc`	4-bit MSE	4-bit	3.8×	+2.71%	Recommended default
`turboquant_k3v4_nc`	3-bit MSE	4-bit	~3.5×	+10.63%	Models ≥8B only
`turboquant_3bit_nc`	3-bit MSE	3-bit	4.9×	+20.59%	Max compression, use carefully

Table 3: vLLM TurboQuant presets

Naming convention: "k8v4" = FP8 keys + 4-bit values. "4bit" = both at 4-bit. "_nc" = with Normalisation Constant — the per-vector L2 norm stored separately (16 bits per token per head), so the quantized representation only encodes direction. Storing the norm separately preserves accuracy at lower bit widths but incurs a small overhead. Without _nc, the magnitude is also quantized, which loses fidelity at 3-bit and below.

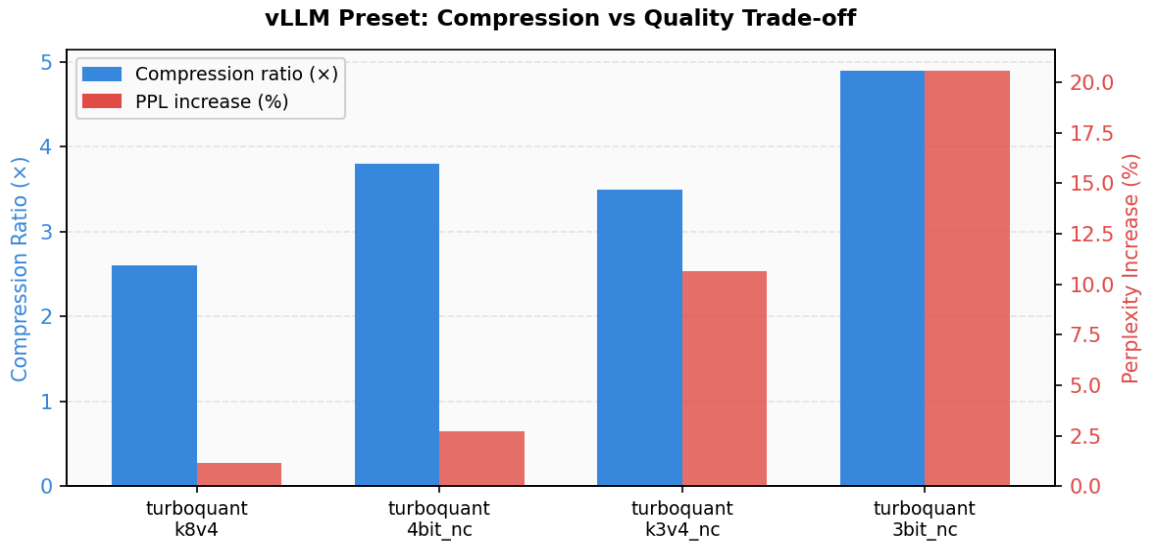

Figure 3: vLLM preset trade-offs — compression ratio vs perplexity increase

5.3 Serving via vLLM CLI

Standard vLLM (Linux / CUDA)

Standard vLLM uses the --kv-cache-dtype flag with named presets. Start with turboquant_k8v4 (safest), then move to turboquant_4bit_nc once quality is validated:

```
# Recommended: 4-bit, ~3.8x compression
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --kv-cache-dtype turboquant_4bit_nc \
  --gpu-memory-utilization 0.95

# Conservative / near-lossless
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --kv-cache-dtype turboquant_k8v4

# Gemma 4 MoE -- use 4bit, not 3bit
vllm serve google/gemma-4-26B-A4B-it \
  --kv-cache-dtype turboquant_4bit_nc \
  --gpu-memory-utilization 0.92
```

vllm-mlx (Apple Silicon -- M-series Macs)

TurboQuant KV cache quantization on Apple Silicon requires the vllm-mlx nightly build -- not the stable release -- and a Python 3.12 virtual environment. Confirmed working on M5 MacBook Pro with unsloth/Qwen3.6-27B-MLX-8bit + 4-bit KV cache quantization. The 8-bit weight model is the recommended choice for demonstrating TurboQuant: higher-quality weights yield a cleaner KV cache compression baseline, with no weight-quantization artifacts to confound the results. For full environment setup, including Homebrew, shell config, and troubleshooting, see Appendix A.

Full setup guide: If you are starting fresh on Apple Silicon, go to Appendix A first. It covers arm64 Homebrew, Python 3.12 installation, venv creation, shell fixes, and a complete troubleshooting table. This section covers only the serve commands.

Step 1: Create a Python 3.12 virtual environment

vllm-mlx supports Python 3.10–3.13; Python 3.14 has no MLX wheels yet. Python 3.12 is recommended as the most stable tested version on M-series Macs.

```
# Use explicit brew-managed arm64 Python 3.12 (not system python3.12)
$(brew --prefix python@3.12)/bin/python3.12 -m venv ~/Develop/vllm-mlx
source ~/Develop/vllm-mlx/bin/activate

# Verify before installing anything
arch                    # must return: arm64
python --version        # must show: Python 3.12.x
python -m pip debug --verbose 2>/dev/null | grep arm64 | head -1
```

Step 2: Install the vllm-mlx nightly build

The --pre flag pulls the pre-release nightly wheel from the vLLM nightly index. The stable PyPI release does not include TurboQuant KV cache quantization support for MLX.

```
pip install vllm-mlx --pre --extra-index-url https://wheels.vllm.ai/nightly
```

Important: You must use --pre together with --extra-index-url https://wheels.vllm.ai/nightly. The stable vllm-mlx release on PyPI does not include the --kv-cache-quantization flags.

Step 3: Verify available flags for your build

Nightly builds change frequently. Always confirm the exact flag names in your installed version before serving:

```
vllm-mlx serve --help | grep -i "cache"
```

Step 4: Serve with KV cache quantization

The vllm-mlx nightly build uses a different flag interface from standard vLLM. The --kv-cache-dtype preset argument is NOT recognised; you must use two flags together: a master toggle and a bit-width argument. Using only --kv-cache-quantization-bits without the toggle causes an "unrecognized arguments" error.

```
# Correct syntax for vllm-mlx nightly builds
# --kv-cache-quantization is the MASTER SWITCH (required first)
```

```
# --kv-cache-quantization-bits sets the bit-width
vllm-mlx serve unsloth/Qwen3.6-27B-MLX-8bit \
  --kv-cache-quantization \
  --kv-cache-quantization-bits 4 \
  --port 8001 \
  --reasoning-parser qwen3

# Optional: reduce group-size to 32 if accuracy degrades (default: 64)
vllm-mlx serve unsloth/Qwen3.6-27B-MLX-8bit \
  --kv-cache-quantization \
  --kv-cache-quantization-bits 4 \
  --kv-cache-quantization-group-size 32 \
  --port 8001
```

Flag reference for vllm-mlx:

- --kv-cache-quantization Master enable switch. Without this, the bit-width flag is silently ignored or rejected.
- --kv-cache-quantization-bits 4 Sets precision. 4-bit recommended; 3-bit degrades quality on models below 8B.
- --kv-cache-quantization-group-size Default 64. Try 32 on M-series if accuracy issues appear (small memory cost).
- To verify RAM savings on Mac: Activity Monitor > Memory tab, compare before and after enabling the flags.

5.4 Python API (HuggingFace-compatible)

For non-vLLM workloads — such as HuggingFace generate loops or custom inference code — install turbokv (the production community implementation of Algorithm 1 from the paper). It is a one-line drop-in that transparently patches any HuggingFace model.

```
# Install the community TurboQuant implementation
pip install turbokv
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from turbokv import patch_model_for_turbokv

MODEL_ID = "meta-llama/Llama-3.1-8B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(MODEL_ID, torch_dtype="auto",
device_map="auto")

# One-line drop-in: patches the model to use TurboQuant KV cache
patch_model_for_turbokv(model, bits=4)    # 4-bit recommended

# model.generate() works exactly as before -- compression is transparent
inputs = tokenizer("Your prompt here", return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=512)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

turbokv implements Algorithm 1 (TurboQuant_mse) from the paper: MSE-only, no QJL. Confirmed distortion of 0.0093 at 4-bit matching the paper bound. Source: github.com/vivekvar-dl/turboquant

5.5 Measuring Impact

Always benchmark before and after enabling TurboQuant. The metrics differ by platform:

CUDA / Linux (nvidia-smi)

```
# Baseline KV cache memory
nvidia-smi dmon -s mu -d 1   # live VRAM monitor (MB used)

# Throughput benchmark with vLLM
python -m vllm.entrypoints.benchmark_throughput \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --kv-cache-dtype turboquant_4bit_nc \
  --input-len 8192 --output-len 512 --num-prompts 50
```

Apple Silicon (Activity Monitor / powermetrics)

```
# GPU + ANE power and memory in terminal
sudo powermetrics --samplers gpu_power -i 1000 -n 5

# Or: Activity Monitor > Memory tab, watch "Memory Used" and "Swap Used"
# Compare with and without --kv-cache-quantization flags

# Unified memory pressure check
vm_stat | grep "Pages active\|Pages wired\|Pages occupied"
```

Output quality (both platforms)

```
# Quick perplexity check with turbokv
pip install turbokv lm-eval

# Run a short LongBench-style eval before and after
lm_eval --model hf --model_args pretrained=Llama-3.1-8B \
  --tasks hellaswag,arc_easy --limit 100
```

6. Gotchas & Known Issues

These are issues identified by 5+ independent community implementation teams — important practical knowledge not emphasised in the paper.

6.1 QJL Hurts in Practice

The paper's second stage (QJL) is mathematically correct for raw inner products but counterproductive for attention. The softmax function exponentially amplifies variance: QJL's random noise gets magnified, while MSE-only quantization has biased but lower-variance inner products — and lower variance wins after softmax.

Action: Use PolarQuant with MSE-only residuals. Skip QJL for all KV cache applications. QJL remains valid for vector-search use cases where no softmax is applied.

6.2 Hybrid Models (Mamba, GDN, Sliding-Window)

vLLM requires all KV cache specs in a cache group to have compatible page sizes. TurboQuant uses uint8 slots of 95 bytes per token per head (vs bf16's 256 bytes), which cannot be reconciled with Mamba state cache pages. Models that mix attention and linear-attention (e.g., Qwen3.6-35B-A3B variants with hybrid GDN/attention layers) will fail with a NotImplementedError on the standard vLLM path.

Action: Avoid TurboQuant on pure hybrid models for now. Standard GQA/MHA architectures (Llama, Qwen, Mistral, Gemma) work correctly. vLLM auto-skips non-decoder layers.

6.3 Value Quantization Quality Floor

Value quantization is the quality bottleneck. At 2-bit values, cosine similarity with the FP16 baseline drops to 0.94 — visible degradation in output quality. At 4-bit values, cosine similarity is 0.997, essentially indistinguishable. Always set value bits equal to or higher than key bits.

6.4 Prefill Phase Behaviour

TurboQuant compresses K and V vectors as they are written to cache, layer-by-layer, during prefill. The compressed cache is what survives into the decode phase. Two practical implications:

- Prefill compute is slightly slower (~5-10% overhead) because each cache write now includes the rotation and quantization step.
- Intermediate activations, not the cache, determine peak memory during prefill. Memory savings appear during decode and across long accumulated sequences — exactly where KV cache dominates.
- Workloads that are prefill-heavy (one-shot Q&A on short prompts) see less benefit. Workloads that are decode-heavy or have long accumulated context (agentic loops, long-form generation, multi-turn chat) see the full advertised compression benefit.

6.5 Apple Silicon (vllm-mlx) — Key Differences

Users running vllm-mlx on Apple Silicon should note these platform-specific behaviours that differ from CUDA vLLM:

- Use --kv-cache-quantization + --kv-cache-quantization-bits 4 instead of --kv-cache-dtype presets. The CUDA preset names (turboquant_4bit_nc, etc.) are not recognised on MLX.
- Unified memory: there is no GPU/CPU memory split. KV cache compression reduces pressure on shared RAM, directly increasing the number of concurrent contexts that fit before macOS starts swapping.
- --gpu-memory-utilization is a CUDA concept. It has no equivalent and should not be passed to vllm-mlx.
- Monitoring: use Activity Monitor (Memory tab) or sudo powermetrics --samplers gpu_power instead of nvidia-smi.
- The nightly build is required — the stable PyPI release does not include KV cache quantization. See Appendix A for full setup.

6.6 Things That Work Well

- No calibration data needed — works on any transformer model out of the box
- Auto-skips incompatible layers — vLLM falls back to FlashAttention on non-decoder layers
- Stacks independently with weight quantization (AWQ, GPTQ, GGUF)
- Compression ratio is hardware-efficient: aligns with H100 tensor core memory access patterns
- Mathematically provable bounds — unlike heuristic pruning methods, distortion has guaranteed limits

7. Structured Learning Path

A practical three-week programme to go from zero to production-confident with TurboQuant.

Week	Focus	Key Tasks
Week 1	**Foundations**	KV cache memory math; read arXiv:2504.19874 (Sections 2-3); run validate_paper.py from 0xSero/turboquant
Week 2	**Hands-on**	Enable turboquant_4bit_nc in vLLM; run LongBench & Needle evals; profile with torch.profiler
Week 3	**Production**	Combine with AWQ/GPTQ weight quantization; test on your real workload; handle caveats for hybrid models.

7.1 Week 1 — Foundations

1. Calculate your KV cache memory footprint.

Formula: layers × seq_len × 2 × hidden_dim × 2 bytes × num_users. For a 70B model at 128k context with 10 concurrent users: 80 × 128000 × 2 × 8192 × 2 × 10 = ~419 GB. That's your motivation number.

2. Read the original paper (arXiv:2504.19874).

Focus on Sections 2 (problem formulation) and 3 (algorithm). Skip Section 4 (vector search QJL applications) unless you have a retrieval use case. The paper is 18 pages; Sections 2-3 are sufficient for practitioners.

3. Run the community validation suite.

Clone 0xSero/turboquant and run validate_paper.py (tests all 3 theorems) and audit_claims.py (adversarial audit of benchmark claims). This builds intuition for where the algorithm works and where it breaks down.

7.2 Week 2 — Hands-On Implementation

4. Enable TurboQuant in vLLM and measure your workload.

Start with turboquant_k8v4, then move to turboquant_4bit_nc. Compare tokens/second, memory usage (nvidia-smi dmon on CUDA; Activity Monitor on Apple Silicon), and output quality on prompts representative of your actual use case — not just LongBench.

5. Run LongBench and Needle-in-Haystack evaluations.

Compare your numbers against the paper's reported baselines. If your numbers are significantly worse, suspect a misconfiguration (the wrong head_dim, QJL accidentally enabled, or a hybrid model architecture issue).

6. Profile attention layer timing.

On CUDA, use torch.profiler to measure per-layer attention time — expect a 3-8× speedup specifically in attention logit computation. On Apple Silicon / vllm-mlx: use Instruments (Xcode > Open Developer Tool > Instruments > Time Profiler), or compare end-to-end tokens/second via the vllm-mlx server logs before and after enabling the quantization flags.torch.profiler does not profile MLX operations.

7.3 Week 3 — Production & Advanced

7. Combine with weight quantization.

TurboQuant (KV cache) and AWQ/GPTQ (weights) stack independently. A 70B model with 4-bit weights + turboquant_4bit_nc can serve 128k-token contexts on 2× H100 that previously required 4×. This is the most impactful combination for cost reduction.

8. Understand the deeper mathematics.

TurboQuant is the scalar case of HIGGS quantization (Malinovskii et al., NAACL 2025, arXiv:2411.17525). Reading that paper gives a complete mathematical foundation via the Linearity Theorem, which explains why random Hadamard rotation enables near-optimal quantization across transformer architectures.

9. Monitor and iterate on production.

Set up perplexity drift monitoring in production — watch for output quality degradation over time as context windows grow. TurboQuant is stable at 4-bit, but the community is still characterising the interaction with extremely long sequences (500k+ tokens).

8. Key References

- **TurboQuant paper:** "Online Vector Quantization with Near-optimal Distortion Rate" — arXiv:2504.19874, ICLR 2026 (Zandieh, Daliri, Hadian, Mirrokni et al.) — arxiv.org/abs/2504.19874
- **HIGGS (mathematical foundation):** "Pushing the Limits of LLM Quantization via the Linearity Theorem" — arXiv:2411.17525, Malinovskii et al., NAACL 2025 — arxiv.org/abs/2411.17525
- **KIVI (prior KV cache method):** "A Tuning-Free Asymmetric 2-bit Quantization for KV Cache" — Liu et al., ICML 2024 — arxiv.org/abs/2402.02750
- **vLLM upstream PR #38479:** TurboQuant KV cache quantization merged into stock vLLM (April 15, 2026) — github.com/vllm-project/vllm/pull/38479
- **Community implementation (validation):** 0xSero/turboquant — includes validate_paper.py and audit_claims.py test suites — github.com/0xSero/turboquant
- **turbokv (production Python package):** vivekvar-dl/turboquant — pip install turbokv — implements Algorithm 1 (TurboQuant_mse), confirmed 0.0093 MSE at 4-bit matching paper bound — github.com/vivekvar-dl/turboquant
- **Google Research blog:** "TurboQuant: Redefining AI efficiency with extreme compression" — research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression

APPENDIX A

vllm-mlx Complete Setup and Troubleshooting Guide

Local LLM Serving on Apple Silicon · M5 MacBook Pro 128 GB · April 2026

A.1 Prerequisites

Before starting, confirm your terminal is running natively on arm64 — not under Rosetta 2. Everything that follows assumes a native arm64 shell.

```
arch
# Must return: arm64
```

Check which Homebrews are installed and which Python versions are present:

```
which brew          # shows active brew
brew --prefix       # /usr/local = Intel   /opt/homebrew = arm64
ls /opt/homebrew/bin/brew   # confirm arm64 brew exists

# Check architecture of each Python 3.12 found on the system
find /usr /opt /Library -name "python3.12" -type f 2>/dev/null | while read p; do
  echo "$p: $(file $p | grep -o 'arm64\|x86_64')"
done
```

If Python shows x86_64 on an M-series Mac, everything pip installs — including MLX — will fail silently. MLX only ships arm64 wheels.

A.2 Fix arm64 Homebrew

Apple Silicon Macs can have two side-by-side Homebrew installations:

Homebrew	Location	Architecture
Intel (Rosetta)	`/usr/local`	x86_64
Apple Silicon	`/opt/homebrew`	**arm64 (required)**

If /usr/local/bin/brew wins in your PATH, every brew install silently produces Intel binaries. Fix ~/.zprofile to initialise arm64 Homebrew first:

```
# ~/.zprofile -- HOMEBREW section

# Initialise arm64 Homebrew FIRST (must come before BREW_PREFIX)
if [[ -x /opt/homebrew/bin/brew ]]; then
  eval "$(/opt/homebrew/bin/brew shellenv)"
fi
```

```
# Now brew --prefix correctly returns /opt/homebrew
export BREW_PREFIX=$(brew --prefix 2>/dev/null)
```

If /opt/homebrew does not exist, install arm64 Homebrew first:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Reload and verify:

```
source ~/.zprofile
which brew        # /opt/homebrew/bin/brew
brew --prefix     # /opt/homebrew
```

A.3 Install arm64 Python 3.12

vllm-mlx supports Python 3.10–3.13. Python 3.14 has no MLX wheels. Python 3.12 is the most stable choice and has been confirmed to work with Qwen3.6-27B on M5.

```
brew install python@3.12

# Confirm it is arm64 -- must say: Mach-O 64-bit executable arm64
file $(brew --prefix python@3.12)/bin/python3.12
```

If the output says x86_64, Homebrew is still resolving to the Intel installation. Return to step A.2 and fix ~/.zprofile before continuing.

A.4 Create the Virtual Environment

Always use an isolated venv. Never install ML packages into global Python.

```
cd ~/Develop

# Create venv using arm64 Python 3.12 explicitly
$(brew --prefix python@3.12)/bin/python3.12 -m venv vllm-mlx

# Activate it -- prompt will show (vllm-mlx)
source ~/Develop/vllm-mlx/bin/activate
```

Verify everything is correct before installing anything:

```
arch                  # arm64
python --version      # Python 3.12.x
which python          # ~/Develop/vllm-mlx/bin/python
```

```
which pip              # ~/Develop/vllm-mlx/bin/pip

# Confirm arm64 platform tags -- must show cp312-cp312-macosx_XX_0_arm64
python -m pip debug --verbose 2>/dev/null | grep arm64 | head -3
```

If grep arm64 returns no matches, the venv was built with an x86_64 interpreter. Delete it and rebuild from step A.3.

A.5 Install vllm-mlx

Install the nightly build with audio extras. The --pre flag is required to pull the pre-release nightly wheel from the vLLM nightly wheel index. The stable PyPI release does not include TurboQuant KV cache quantization support.

```
# Nightly build -- required for KV cache quantization support
pip install vllm-mlx --pre --extra-index-url https://wheels.vllm.ai/nightly

# Optional: audio extras (TTS + STT support)
pip install 'vllm-mlx[audio]' --pre --extra-index-url
https://wheels.vllm.ai/nightly

# Required for non-English text-to-speech
brew install espeak-ng

# Verify
vllm-mlx --version
```

If you plan to use gated models (e.g. Llama-3 family), set your HuggingFace token before first use:

```
# Set HF token (required for gated models like Llama-3)
pip install huggingface_hub
huggingface-cli login

# Or set as environment variable
export HF_TOKEN=hf_your_token_here
# Add to ~/.zprofile to persist across sessions
```

zsh quoting note: The square brackets in 'vllm-mlx[audio]' must be single-quoted. Without quotes, zsh interprets [audio] as a glob pattern and throws: zsh: no matches found.

A.6 Fix zshrc — Shell Config

Remove hardcoded pip3 alias

If your ~/.zshrc has a hardcoded pip3 alias pointing to a specific Python, remove it — it bypasses the venv and causes "externally managed environment" errors:

```
# REMOVE THIS from ~/.zshrc
# alias pip3="$BREW_PREFIX/opt/python@3.14/bin/python3 -m pip"

# Replace with:
# pip3 intentionally not aliased -- use pip inside your activated venv
```

Fix the chpwd() auto-activate hook

If you have a chpwd() auto-activate hook, ensure: (1) the path guard uses the correct capitalisation of your Develop folder, and (2) venv detection uses a dynamic find instead of a hardcoded .venv name.

```
chpwd() {
  # Deactivate when leaving ~/Develop -- note capital D
  if [[ -n "$VIRTUAL_ENV" && "$PWD" != "$HOME/Develop"* ]]; then
    deactivate && echo "venv deactivated"
  fi
  [[ "$PWD" != "$HOME/Develop"* ]] && return

  # Auto-activate any venv one level deep -- no hardcoded names
  local venv_activate
  venv_activate=$(find . -mindepth 2 -maxdepth 2 -name activate \
    -path "*/bin/activate" 2>/dev/null | head -1)
  if [[ -n "$venv_activate" && -z "$VIRTUAL_ENV" ]]; then
    source "$venv_activate"
    echo "venv activated ($(basename $(dirname $(dirname $venv_activate))))"
  fi

  # Auto-switch Node version if .nvmrc exists
  if [[ -f .nvmrc ]]; then
    nvm use
  fi
}
```

mindepth 2 -maxdepth 2 matches ./vllm-mlx/bin/activate exactly — it will not descend into sibling project directories.

A.7 Serve the Model

The model downloads automatically on first run. Cached at ~/.cache/huggingface/hub/ and loads instantly on subsequent runs.

```
# Activate venv
source ~/Develop/vllm-mlx/bin/activate

# Serve Qwen3.6-27B 8-bit with TurboQuant KV cache compression
vllm-mlx serve unsloth/Qwen3.6-27B-MLX-8bit \
  --kv-cache-quantization \
  --kv-cache-quantization-bits 4 \
  --port 8001 \
  --reasoning-parser qwen3
```

```
# Without KV cache compression (baseline comparison)
vllm-mlx serve unsloth/Qwen3.6-27B-MLX-8bit \
  --port 8001 \
  --reasoning-parser qwen3

# Wait for: Server started on 0.0.0.0:8001
```

Model	Size	Use for
unsloth/Qwen3.6-27B-MLX-8bit	**34.7 GB**	**Recommended — cleanest TurboQuant baseline**
unsloth/Qwen3.6-27B-UD-MLX-4bit	26.2 GB	Lighter option — smaller RAM footprint
mlx-community/Qwen3.6-27B-4bit	~26 GB	Standard uniform quant alternative

CUDA-only flags — DO NOT use on Apple Silicon:--kv-cache-dtype turboquant_4bit_nc (requires CUDA Triton kernels)--gpu-memory-utilization 0.95 (VRAM concept, not applicable to unified memory)--enable-mllm-cache / --mllm-cache-max-mb (not available in vllm-mlx 0.2.x)

A.8 Test with curl

Run these in a second terminal tab while the server is running on port 8001.

Health check

```
curl http://localhost:8001/health
# Expected (varies by vllm-mlx version):
#   {"status": "healthy"}                                -- minimal
#   {"status": "healthy", "model_loaded": true,          -- detailed
#    "model_type": "mllm"}
# If model_type is reported, mllm = vision active, llm = text-only.
# A 200 OK status is the only universally reliable signal.
```

Text inference

```
curl -sS http://localhost:8001/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "unsloth/Qwen3.6-27B-MLX-8bit",
    "prompt": "<|im_start|>user\nExplain vLLM in one
paragraph.<|im_end|>\n<|im_start|>assistant\n<think>\n\n</think>\n\n",
    "max_tokens": 512,
    "temperature": 0,
    "stop": ["<|im_end|>", "<|im_start|>"]
  }'
```

Vision — send an image

```
curl -sS http://localhost:8001/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "unsloth/Qwen3.6-27B-MLX-8bit",
    "messages": [{"role": "user", "content": [
      {"type": "image_url", "image_url": {"url": "https://example.com/photo.jpg"}},
      {"type": "text", "text": "What is in this image?"}
    ]}]
  }'
```

Thinking mode

```
curl -sS http://localhost:8001/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "unsloth/Qwen3.6-27B-MLX-8bit",
    "thinking": true,
    "messages": [{"role": "user", "content": "What is 17 x 43?"}]
  }'
# A reasoning field appears alongside content in the response.
```

A.9 Shell Wrapper — qwen()

Add this function to ~/.zshrc for a quick CLI interface. It applies the Qwen chat template manually and strips leaked <think> blocks from the output. Requires jq.

```
which jq || brew install jq
```

```
qwen() {
  local user_prompt="$*"
  local endpoint="http://127.0.0.1:8001/v1/completions"
  local model="unsloth/Qwen3.6-27B-MLX-8bit"
  curl --fail-with-body -sS --connect-timeout 5 --max-time 300 \
    "$endpoint" -H "Content-Type: application/json" \
    -d "$(jq -n --arg model \"$model\" --arg p \"$user_prompt\" \
      '{model: $model, max_tokens: 1024, temperature: 0,
       prompt: "<|im_start|>user\n" + $p +
"<|im_end|>\n<|im_start|>assistant\n<think>\n\n</think>\n\n",
       stop: ["<|im_end|>","<|im_start|>"]}') \
  | jq -r '.choices[0].text'
  | perl -0pe "s/<think>.*?<\/think>\s*//gs"
}

# Usage after: source ~/.zshrc
qwen "Explain vLLM in one paragraph."
qwen "Give me 5 cybersecurity use cases for a local LLM on Apple Silicon."
```

The empty <think></think> block in the prompt tells the model to skip visible reasoning and go straight to the answer. The perl pipe strips any thinking that leaks through, regardless.

A.10 Python Client

Save as qwen_local.py in your project directory. Handles text inference, thinking-block stripping, and vision (image URL) inputs.

```
pip install requests
```

```
import re, sys, requests

BASE_URL = "http://127.0.0.1:8001"
MODEL    = "unsloth/Qwen3.6-27B-MLX-8bit"

def strip_thinking(text):
    return re.sub(r"<think>.*?</think>\s*", "", text, flags=re.DOTALL).strip()

def qwen(prompt, max_tokens=1024, temperature=0.0):
    fmt = ("<|im_start|>user\n" + prompt +
           "<|im_end|>\n<|im_start|>assistant\n<think>\n\n</think>\n\n")
    r = requests.post(f"{BASE_URL}/v1/completions", json={
        "model": MODEL, "prompt": fmt,
        "max_tokens": max_tokens, "temperature": temperature,
        "stop": ["<|im_end|>", "<|im_start|>"]
    }, timeout=(5, 300))
    r.raise_for_status()
    return strip_thinking(r.json()["choices"][0]["text"])

def qwen_vision(prompt, image_url, max_tokens=1024):
    r = requests.post(f"{BASE_URL}/v1/chat/completions", json={
        "model": MODEL,
        "messages": [{"role": "user", "content": [
            {"type": "image_url", "image_url": {"url": image_url}},
            {"type": "text", "text": prompt}
        ]}], "max_tokens": max_tokens
    }, timeout=(5, 300))
    r.raise_for_status()
    return r.json()["choices"][0]["message"]["content"]

if __name__ == "__main__":
    print(qwen(" ".join(sys.argv[1:]) or "Explain vLLM."))
```

A.11 Quick Reference

Daily workflow

```
# Terminal tab 1 -- start the server
source ~/Develop/vllm-mlx/bin/activate
vllm-mlx serve unsloth/Qwen3.6-27B-MLX-8bit \
  --kv-cache-quantization \
  --kv-cache-quantization-bits 4 \
  --port 8001 \
  --reasoning-parser qwen3

# Terminal tab 2 -- query it
```

```
qwen "your question here"
```

Useful diagnostics

```
# Model cache size
du -sh ~/.cache/huggingface/hub/

# Watch download progress on first run (install watch first on macOS)
brew install watch   # one-time, if not already installed
watch -n 2 'du -sh ~/.cache/huggingface/hub/'

# Or use macOS-native alternative:
while true; do du -sh ~/.cache/huggingface/hub/ ; sleep 2; done

# Check active ports
lsof -i -P -n | grep LISTEN

# Full environment verification
arch && which brew && brew --prefix && python --version
curl http://localhost:8001/health

# Confirm arm64 platform tags
python -m pip debug --verbose 2>/dev/null | grep arm64 | head -3
```

A.12 Troubleshooting

Error	Cause	Fix
`zsh: no matches found: vllm-mlx[audio]`	zsh glob on brackets	Single-quote it: 'vllm-mlx[audio]'
`mlx has no matching distributions`	Wrong Python arch or version	Rebuild venv with arm64 Python 3.12
`externally-managed-environment`	pip3 alias bypassing venv	Remove hardcoded pip3 alias from .zshrc
`unrecognized arguments: --kv-cache-dtype`	CUDA-only flag	Remove — not supported on Apple Silicon
`unrecognized arguments: --gpu-memory-utilization`	CUDA-only flag	Remove — not applicable to unified memory
`model_type: llm in health check`	Vision encoder not loaded	Check the model name or use the mlx-vlm compatible model
`venv deactivates on cd`	chpwd() capitalisation bug	Fix 'develop' to 'Develop' in path guard
`venv not auto-activating`	Hook only checks .venv	Use find -mindepth 2 -maxdepth 2 dynamic scan
`Compatible tags show x86_64`	Intel Homebrew Python	Rebuild from /opt/homebrew/opt/python@3.12/bin/python3.12